1.1 Pembersihan Teks (Text Cleaning)

Pembersihan teks meliputi serangkaian transformasi untuk menghilangkan noise dan menormalkan representasi teks. Berikut adalah komponen utamanya.

1.1.1 Tokenisasi (Tokenization)

Tokenisasi adalah proses memecah string teks menjadi unit-unit yang lebih kecil yang disebut token. Token dapat berupa kata, sub-kata (subword), atau karakter, tergantung pada granularitas yang dibutuhkan.

- **Word tokenization:** memecah kalimat menjadi kata berdasarkan whitespace dan tanda baca. Contoh: *"Saya makan nasi." → ["Saya", "makan", "nasi", "."]*
- **Subword tokenization:** memecah kata menjadi unit yang lebih kecil (BPE, WordPiece, SentencePiece). Berguna untuk menangani out-of-vocabulary (OOV) words.
- **Sentence tokenization:** memecah dokumen menjadi kalimat-kalimat berdasarkan tanda baca dan aturan linguistik.

Implementasi cepat dengan NLTK:

```
from nltk.tokenize import word_tokenize, sent_tokenize
tokens = word_tokenize("Saya belajar NLP hari ini.")
# Output: ['Saya', 'belajar', 'NLP', 'hari', 'ini', '.']
```

1.1.2 Lowercasing

Pengubahan teks menjadi huruf kecil bertujuan untuk menyamakan kata dengan kapitalisasi berbeda agar dianggap identik oleh model. Tanpa lowercasing, kata "Indonesia" dan "indonesia" akan dipetakan ke vektor yang berbeda.

```
text = "Saya Belajar NLP".lower()  # "saya belajar nlp"
```

Catatan penting: lowercasing tidak selalu disarankan untuk task seperti Named Entity Recognition (NER) karena kapitalisasi adalah sinyal penting untuk mengidentifikasi nama entitas.

1.1.3 Penghapusan Stopword (Stopword Removal)

Stopword adalah kata-kata yang sangat sering muncul namun memiliki muatan informasi yang rendah, seperti "yang", "di", "dan", "ini" dalam Bahasa Indonesia, atau "the", "is", "at" dalam Bahasa Inggris. Penghapusan stopword mengurangi dimensi fitur dan membantu model fokus pada kata bermakna.

```
from nltk.corpus import stopwords
stop_id = set(stopwords.words('indonesian'))
```

```
filtered = [w for w in tokens if w.lower() not in stop_id]
```

Peringatan: untuk task seperti analisis sentimen, kata seperti "tidak" atau "bukan" sangat penting; jangan hapus stopword secara membabi buta tanpa mempertimbangkan konteks task.

1.1.4 Penghapusan Karakter Khusus dan Tanda Baca

Karakter khusus seperti emoji, simbol HTML, URL, mention, hashtag, dan tanda baca biasanya dihapus menggunakan regular expression (regex).

```
import re
text = re.sub(r'http\S+', '', text)         # hapus URL
text = re.sub(r'@\w+', '', text)            # hapus mention
text = re.sub(r'[^a-zA-Z0-9\s]', '', text)  # hapus non-alfanumerik
text = re.sub(r'\s+', ' ', text).strip()    # normalisasi whitespace
```

1.3 Pipeline Pra-pemrosesan Standar

Urutan tahapan yang umum dilakukan dalam pipeline pra-pemrosesan adalah sebagai berikut.

1. Lowercasing seluruh teks.
2. Penghapusan URL, mention, hashtag, dan elemen non-tekstual lainnya.
3. Penghapusan tanda baca dan karakter khusus.
4. Tokenisasi menjadi kata.
5. Penghapusan stopword.
6. Stemming atau lemmatization.
7. Output: list token bersih yang siap diproses tahap ekstraksi fitur.

1.2 Stemming

Stemming adalah proses memotong kata ke bentuk akar (root form) dengan menghilangkan imbuhan (afiks) seperti awalan, sisipan, akhiran, dan konfiks. Tujuannya adalah agar kata-kata yang berasal dari akar yang sama dipetakan ke representasi yang seragam, sehingga mengurangi sparsity vektor fitur.

1.2.1 Konsep Dasar

- **Input:** kata berimbuhan, contoh "berlarian", "pelarian", "melarikan".
- **Output:** akar kata, yaitu "lari".
- **Pendekatan:** berbasis aturan (rule-based) yang mendefinisikan pola imbuhan, atau berbasis kamus untuk validasi.

1.2.2 Stemming vs Lemmatization

Keduanya bertujuan menormalkan kata, namun memiliki pendekatan dan hasil yang berbeda.

Aspek	Stemming	Lemmatization
Pendekatan	Aturan pemotongan afiks	Analisis morfologis dengan kamus (lemma)
Kecepatan	Cepat	Lebih lambat
Hasil	Bisa bukan kata valid (contoh: "studies" → "studi")	Selalu kata valid (contoh: "studies" → "study")
Konteks	Tidak mempertimbangkan POS tag	Mempertimbangkan POS tag dan konteks
Pustaka populer	Porter, Snowball, Sastrawi (ID)	WordNetLemmatizer (NLTK), spaCy

Bab 2 — Ekstraksi Fitur

Setelah teks dibersihkan, langkah berikutnya adalah mengubah teks menjadi representasi numerik (vektor) agar dapat diproses oleh algoritma machine learning. Tahap ini disebut feature extraction atau text vectorization.

2.1 Bag-of-Words (BoW)

Bag-of-Words memetakan setiap dokumen menjadi vektor numerik yang nilainya merepresentasikan frekuensi kemunculan kata dalam dokumen tersebut. Urutan kata diabaikan; yang dipertahankan hanyalah informasi tentang kata apa saja yang muncul dan berapa kali.

2.1.1 Cara Kerja

8. Bangun kosakata (vocabulary) dari seluruh kata unik di korpus.
9. Untuk setiap dokumen, hitung frekuensi setiap kata dalam vocabulary.
10. Representasikan dokumen sebagai vektor berukuran |V|, di mana |V| adalah ukuran vocabulary.

2.1.2 Ilustrasi

Diberikan dua dokumen:

- D1: "saya suka kopi"
- D2: "saya suka teh dan kopi"

Vocabulary: `[saya, suka, kopi, teh, dan]`

Dokumen	saya	suka	kopi	teh	dan
D1	1	1	1	0	0
D2	1	1	1	1	1

2.1.3 Implementasi

```
from sklearn.feature_extraction.text import CountVectorizer
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)
print(vectorizer.get_feature_names_out())
```

2.1.4 Kelebihan dan Kekurangan

- **Kelebihan:** sederhana, mudah diimplementasikan, dan baseline yang kuat untuk task klasifikasi sederhana.
- **Kekurangan:** vektor sangat sparse untuk korpus besar, urutan kata hilang, dan tidak mempertimbangkan importance kata.

2.2 N-Grams

N-Gram adalah sekuen n token berurutan yang diambil dari teks. Penggunaan n-gram memungkinkan model menjaga sebagian urutan dan konteks lokal yang hilang pada BoW murni.

2.2.1 Jenis-jenis N-Gram

N	Nama	Contoh dari kalimat "saya suka NLP"
1	Unigram	["saya", "suka", "NLP"]
2	Bigram	[("saya", "suka"), ("suka", "NLP")]
3	Trigram	[("saya", "suka", "NLP")]

2.2.2 Manfaat dan Trade-off

- **Manfaat:** menangkap konteks lokal seperti "tidak bagus" yang artinya berbeda dengan "bagus" pada unigram.
- **Trade-off:** ukuran vocabulary tumbuh eksponensial, dan banyak n-gram muncul sangat jarang sehingga menyebabkan data sparsity.
- **Praktik umum:** kombinasi unigram + bigram (range (1, 2)) sering memberikan keseimbangan terbaik antara coverage dan sparsity.

2.2.3 Implementasi

```
vectorizer = CountVectorizer(ngram_range=(1, 2))  # unigram + bigram
X = vectorizer.fit_transform(corpus)
```

2.3 TF-IDF (Term Frequency – Inverse Document Frequency)

TF-IDF adalah skema pembobotan yang memberikan bobot lebih besar pada kata yang spesifik untuk suatu dokumen, dan menurunkan bobot kata yang muncul di hampir semua dokumen (yang dianggap kurang informatif).

2.3.1 Rumus

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

Term Frequency (TF) mengukur seberapa sering term t muncul dalam dokumen d:

$$TF(t, d) = (jumlah\ kemunculan\ t\ di\ d) \,/\, (total\ term\ di\ d)$$

Inverse Document Frequency (IDF) mengukur seberapa langka term t di seluruh korpus:

$$IDF(t) = log(\, N \,/\, df(t)\,)$$

dengan N = total dokumen di korpus, dan df(t) = jumlah dokumen yang mengandung term t. Beberapa varian menambahkan smoothing untuk menghindari pembagian dengan nol:

$$IDF(t) = log(\,(1 + N) \,/\, (1 + df(t))\,) + 1$$

2.3.2 Intuisi

- **TF tinggi:** term sering muncul di dokumen → relevan untuk dokumen tersebut.
- **IDF tinggi:** term jarang muncul di korpus → diskriminatif antar dokumen.
- **TF-IDF tinggi:** term sering muncul di dokumen ini namun jarang di dokumen lain → kata kunci spesifik.
- **TF-IDF rendah:** term yang umum di mana-mana (mendekati stopword) → bobotnya kecil.

2.3.3 Contoh Perhitungan

Misalkan korpus terdiri dari 3 dokumen, dan term "kopi" muncul di 2 dokumen. Pada dokumen D1 yang memiliki 10 token, kata "kopi" muncul 2 kali.

- TF(kopi, D1) = 2/10 = 0.2
- IDF(kopi) = log(3/2) ≈ 0.176
- TF-IDF(kopi, D1) ≈ 0.2 × 0.176 = 0.0352

2.3.4 Implementasi

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer(ngram_range=(1, 2), max_features=5000)
X = tfidf.fit_transform(corpus)
```

2.4 Perbandingan Singkat

Metode	Mempertahankan Urutan	Pembobotan	Sparsity
BoW	Tidak	Frekuensi mentah	Tinggi
N-Gram	Sebagian (lokal)	Frekuensi n-gram	Sangat tinggi
TF-IDF	Tidak (kecuali + n-gram)	TF × IDF	Tinggi tapi terbobot

$$IDF(w, D) = \ln\left(\frac{Total\ number\ of\ documents\ (N)\ in\ corpus\ D}{number\ of\ documents\ containing\ w}\right)$$

Bab 3 — Pemodelan Bahasa

Language Model (LM) adalah model probabilistik yang memberikan probabilitas pada sekuen kata. Tujuan utamanya adalah memprediksi kata berikutnya berdasarkan konteks sebelumnya, atau memberikan skor likelihood pada suatu kalimat.

3.1 Konsep N-Gram Language Model

N-Gram LM menghitung probabilitas suatu kalimat dengan mendekomposisinya menjadi probabilitas bersyarat antar kata. Probabilitas penuh suatu kalimat $w_1, w_2, \ldots, w_n$ dirumuskan dengan chain rule:

$$P(w_1, w_2, \ldots, w_n) = \Pi\, P(w_i \mid w_1, \ldots, w_{i-1})$$

3.1.1 Markov Assumption

Menghitung probabilitas dengan memperhitungkan seluruh history sebelumnya tidak praktis karena masalah sparsity. Markov assumption menyederhanakan dengan mengasumsikan bahwa probabilitas kata berikutnya hanya bergantung pada n-1 kata sebelumnya saja.

- **Unigram (n=1):** $P(w_i)$ — independen sepenuhnya, tidak mempertimbangkan konteks.
- **Bigram (n=2):** $P(w_i \mid w_{i-1})$ — bergantung pada satu kata sebelumnya.
- **Trigram (n=3):** $P(w_i \mid w_{i-2}, w_{i-1})$ — bergantung pada dua kata sebelumnya.

3.1.2 Estimasi Probabilitas dengan MLE

Dengan Maximum Likelihood Estimation, probabilitas bigram dihitung dari rasio frekuensi:

$$P(w_i \mid w_{i-1}) = count(w_{i-1}, w_i) \,/\, count(w_{i-1})$$

Contoh: dari korpus, jika count("saya makan") = 30 dan count("saya") = 100, maka:

$$P(makan \mid saya) = 30 \,/\, 100 = 0.30$$

3.1.3 Smoothing

MLE memberi probabilitas nol untuk n-gram yang tidak pernah muncul di data training. Ini bermasalah karena akan membuat probabilitas seluruh kalimat menjadi nol. Solusi umum:

- **Laplace (Add-One) Smoothing:** menambahkan 1 ke setiap count.

$$P(w_i \mid w_{i-1}) = (\, count(w_{i-1}, w_i) + 1\,) \,/\, (\, count(w_{i-1}) + |V|\,)$$

- **Add-k Smoothing:** generalisasi dengan menambahkan k < 1.
- **Kneser-Ney Smoothing:** teknik canggih yang sering memberi performa terbaik untuk n-gram LM.
- **Backoff dan Interpolation:** kombinasikan probabilitas dari unigram, bigram, trigram untuk mengatasi sparsity.

3.1.4 Evaluasi: Perplexity

Perplexity adalah metrik standar untuk mengevaluasi LM. Semakin rendah nilainya, semakin baik model dalam memprediksi data uji.

$$PP(W) = P(w_1, w_2, \ldots, w_n)^{(-1/n)}$$

3.2 Fungsi Utama Language Model

Language Model menjadi komponen mendasar di banyak aplikasi NLP modern.

3.2.1 Smart Reply dan Auto-completion

Sistem seperti smart reply pada Gmail atau auto-completion keyboard di smartphone menggunakan LM untuk memprediksi kata atau frasa selanjutnya berdasarkan konteks yang sudah diketik pengguna.

3.2.2 Pengenalan Suara (Automatic Speech Recognition)

Pada ASR, LM digunakan untuk memilih hipotesis transkripsi yang paling probabel di antara beberapa kandidat yang dihasilkan acoustic model. LM membantu disambiguasi kata homofon seperti "rok" vs "rock".

3.2.3 Terjemahan Mesin (Machine Translation)

Pada statistical machine translation klasik (model noisy channel), LM bahasa target memastikan output terjemahan terdengar natural dan gramatikal di bahasa target.

3.2.4 Aplikasi Lain

- Spelling correction dan grammar checking.
- Optical Character Recognition (OCR) — disambiguasi karakter mirip.
- Information retrieval (query likelihood model).
- Text generation untuk chatbot, summarization, dan creative writing.

Bab 4 — Analisis Tata Bahasa dan Entitas

Bab ini membahas dua task fundamental NLP: Part-of-Speech (POS) Tagging untuk melabeli kelas kata, dan Named Entity Recognition (NER) untuk mengekstrak entitas seperti nama orang, lokasi, dan tanggal dari teks.

4.1 POS Tagging (Part-of-Speech Tagging)

POS Tagging adalah proses memberi label kelas kata (noun, verb, adjective, adverb, dst.) pada setiap token dalam kalimat. Output ini menjadi fitur dasar untuk task downstream seperti parsing, NER, dan information extraction.

Contoh: "Saya membaca buku." → [("Saya", PRON), ("membaca", VERB), ("buku", NOUN), (".", PUNCT)]

4.1.1 Pendekatan Rule-Based

Pendekatan ini mengandalkan kamus dan aturan linguistik manual untuk menentukan tag suatu kata berdasarkan morfologi dan konteks sintaktiknya.

- **Kelebihan:** transparan dan dapat dikontrol secara eksplisit oleh ahli linguistik.
- **Kekurangan:** rapuh terhadap variasi bahasa, sulit di-maintain saat aturan bertambah, dan tidak skalabel untuk bahasa dengan morfologi kompleks.

4.1.2 Pendekatan Transformation-Based (Brill Tagger)

Brill Tagger adalah pendekatan hybrid yang awalnya memberikan tag default berdasarkan frekuensi, lalu menerapkan rangkaian aturan transformasi yang dipelajari otomatis dari data berlabel untuk mengoreksi tag awal tersebut.

- **Cara kerja:** learn rules of the form "ubah tag X menjadi Y jika kondisi konteks Z terpenuhi".
- **Kelebihan:** aturan yang dihasilkan dapat dibaca manusia, lebih akurat dari rule-based murni.

4.1.3 Pendekatan Statistik: Hidden Markov Model (HMM)

HMM memodelkan sekuen tag sebagai hidden states dan kata sebagai observations. Model ini memilih sekuen tag yang memaksimalkan probabilitas gabungan.

$$argmax_T P(T \mid W) = argmax_T \Pi P(t_i \mid t_{i-1}) \times P(w_i \mid t_i)$$

Dua probabilitas utama:

- **Transition probability:** $P(t_i \mid t_{i-1})$ — peluang transisi antar tag.
- **Emission probability:** $P(w_i \mid t_i)$ — peluang menghasilkan kata w_i dari tag t_i.

Dekoding (mencari sekuen tag terbaik) dilakukan dengan algoritma **Viterbi** yang efisien menggunakan dynamic programming.

4.1.4 Pendekatan Modern: Neural Tagger

Pendekatan modern menggunakan BiLSTM-CRF atau Transformer-based tagger (BERT) yang memberi hasil state-of-the-art. Model ini mempelajari representasi kontekstual dan mampu menangani out-of-vocabulary lebih baik.

4.2 NER (Named Entity Recognition)

NER adalah task mengekstrak dan mengklasifikasikan entitas bernama (named entities) dalam teks ke kategori-kategori predefined seperti PERSON, ORGANIZATION, LOCATION, DATE, MONEY, dan PERCENT.

Contoh: "Joko Widodo lahir di Surakarta pada 21 Juni 1961."

- **Joko Widodo** → PERSON
- **Surakarta** → LOCATION
- **21 Juni 1961** → DATE

4.2.1 Pendekatan Berbasis Kamus (Dictionary-Based / Gazetteer)

Sistem mencocokkan token pada teks dengan daftar entitas predefined (gazetteer). Contohnya, daftar nama kota Indonesia digunakan untuk mengekstrak entitas LOCATION.

- **Kelebihan:** akurasi tinggi untuk entitas yang ada di kamus.
- **Kekurangan:** tidak dapat mengenali entitas baru yang tidak ada di kamus, dan rentan ambiguitas ("Solo" sebagai kota vs nama orang).

4.2.2 Pendekatan Berbasis Aturan

Menggunakan pola regex dan aturan linguistik. Contoh aturan: "jika dua kata berurutan diawali huruf kapital dan didahului kata 'di', maka kandidat LOCATION".

- **Kelebihan:** efektif untuk pola yang sangat regular seperti tanggal atau mata uang.
- **Kekurangan:** sulit diperluas untuk entitas yang variatif.

4.2.3 Pendekatan Machine Learning

Memodelkan NER sebagai sequence labeling task dengan skema BIO/BIOES.

- **Skema BIO:** `B-PER` = beginning of person, `I-PER` = inside of person, `O` = outside (bukan entitas).
- **Algoritma klasik:** Conditional Random Fields (CRF), HMM, MaxEnt.
- **Algoritma modern:** BiLSTM-CRF, BERT-NER, IndoBERT-NER untuk Bahasa Indonesia.

4.3 Implementasi dengan Pustaka Python

4.3.1 NLTK

NLTK menyediakan POS tagger berbasis Penn Treebank dan NER tagger sederhana.

```
import nltk
from nltk import word_tokenize, pos_tag, ne_chunk

text = "Barack Obama was born in Hawaii."
```

```
tokens = word_tokenize(text)
tagged = pos_tag(tokens)
# [('Barack', 'NNP'), ('Obama', 'NNP'), ('was', 'VBD'), ...]

entities = ne_chunk(tagged)
# Tree dengan node berlabel PERSON, GPE, dst.
```

4.3.2 spaCy

spaCy lebih cepat, akurat, dan production-ready dibanding NLTK. Mendukung POS tagging dan NER dalam satu pipeline.

```
import spacy
nlp = spacy.load('en_core_web_sm')  # atau 'xx_ent_wiki_sm' untuk multibahasa

doc = nlp("Apple is looking at buying U.K. startup for $1 billion")
for token in doc:
    print(token.text, token.pos_, token.tag_)

for ent in doc.ents:
    print(ent.text, ent.label_)
# Apple ORG, U.K. GPE, $1 billion MONEY
```

4.3.3 Perbandingan NLTK vs spaCy

Aspek	NLTK	spaCy
Tujuan	Edukasi dan riset	Produksi
Kecepatan	Lambat	Sangat cepat (Cython)
Akurasi default	Cukup	Tinggi (model neural)
Multi-language	Manual	Banyak built-in models
API	Modular, prosedural	Pipeline objek-oriented

Bab 5 — Klasifikasi Teks

Klasifikasi teks (text classification) adalah task supervised learning untuk menetapkan satu atau lebih label kelas pada dokumen teks. Aplikasinya luas: analisis sentimen, deteksi spam, kategorisasi berita, deteksi hate speech, dan pemfilteran konten.

5.1 Alur Pipeline Klasifikasi Teks

Berikut urutan kerja standar yang harus dihafalkan dan dipahami.

11. **Pengumpulan Data —** kumpulkan dataset berlabel yang merepresentasikan distribusi target. Pastikan kelas seimbang atau lakukan teknik balancing.
12. **Pra-pemrosesan —** lakukan tokenisasi, lowercasing, penghapusan stopword, stemming, dan pembersihan karakter khusus (lihat Bab 1).
13. **Ekstraksi Fitur —** ubah teks menjadi vektor numerik menggunakan BoW, TF-IDF, n-gram, atau embedding (Word2Vec, GloVe, BERT) (lihat Bab 2).
14. **Pelatihan Model —** split data menjadi train/validation/test (umumnya 70/15/15 atau 80/10/10), pilih algoritma, dan latih model dengan tuning hyperparameter.
15. **Evaluasi —** ukur performa pada data uji menggunakan accuracy, precision, recall, F1-score, dan confusion matrix.
16. **Prediksi —** deploy model untuk inferensi pada data baru, dengan pra-pemrosesan dan vectorization yang konsisten dengan tahap training.

5.2 Algoritma: ML Tradisional vs Deep Learning

5.2.1 Algoritma Machine Learning Tradisional

a. Naive Bayes

Berbasis Bayes Theorem dengan asumsi naif bahwa fitur (kata) saling independen. Sangat cepat, baseline kuat untuk klasifikasi teks, terutama dengan TF-IDF.

$$P(c|d) \propto P(c) \times \Pi P(w_i|c)$$

- **Varian:** Multinomial NB (untuk count features), Bernoulli NB (untuk binary features).

b. Logistic Regression

Model linier yang memprediksi probabilitas kelas menggunakan fungsi sigmoid (binary) atau softmax (multi-class). Stabil, mudah diinterpretasi, dan menjadi baseline yang kuat ketika dipasangkan dengan TF-IDF.

c. Support Vector Machine (SVM)

Mencari hyperplane optimal yang memisahkan kelas dengan margin terbesar. Sangat efektif pada ruang fitur berdimensi tinggi seperti TF-IDF text features.

- **Linear SVM:** default untuk text classification, cepat dan akurat.
- **Kernel SVM (RBF, poly):** untuk batas keputusan non-linier; jarang dibutuhkan pada teks.

5.2.2 Algoritma Deep Learning

a. RNN dan LSTM/GRU

Recurrent Neural Network memproses teks secara sekuensial, mempertahankan hidden state yang merepresentasikan konteks. LSTM dan GRU mengatasi masalah vanishing gradient dengan gating mechanism.

- Cocok untuk task yang memerlukan dependensi temporal jangka panjang.
- Bidirectional LSTM (BiLSTM) memproses dari kiri-ke-kanan dan kanan-ke-kiri untuk konteks penuh.

b. CNN untuk Teks

Convolutional Neural Network dengan filter 1D yang menyapu kata-kata, efektif menangkap fitur lokal seperti n-gram secara otomatis. Cepat dilatih dan kompetitif dengan RNN pada banyak task.

c. Transformer dan BERT

Arsitektur Transformer dengan mekanisme self-attention menjadi state-of-the-art. BERT (Bidirectional Encoder Representations from Transformers) di-pretrain pada korpus besar dan di-fine-tune untuk task downstream.

- **Varians:** BERT, RoBERTa, DistilBERT, ALBERT, dan untuk Bahasa Indonesia: IndoBERT, IndoBERTweet.
- **Fine-tuning:** tambahkan classification head di atas [CLS] token dan latih pada data domain.

Bab 6 — Pemodelan Topik

Topic Modeling adalah teknik unsupervised untuk menemukan struktur tematik tersembunyi dalam koleksi dokumen. Setiap dokumen direpresentasikan sebagai campuran topik, dan setiap topik direpresentasikan sebagai distribusi kata.

6.1 LDA (Latent Dirichlet Allocation)

LDA adalah model probabilistik generatif yang mengasumsikan dokumen dihasilkan dari proses berikut: pilih distribusi topik untuk dokumen, lalu untuk setiap kata pilih topik dari distribusi itu, lalu pilih kata dari distribusi kata di topik tersebut.

6.1.1 Asumsi Generatif

17. Setiap dokumen adalah campuran beberapa topik dengan proporsi tertentu.
18. Setiap topik adalah distribusi probabilitas atas vocabulary.
19. Distribusi proporsi topik per dokumen mengikuti Dirichlet dengan parameter α.
20. Distribusi kata per topik mengikuti Dirichlet dengan parameter β.

6.1.2 Hyperparameter

Parameter	Pengaruh	Interpretasi
α (alpha)	Distribusi topik per dokumen	α tinggi → dokumen memuat banyak topik; α rendah → dokumen fokus pada sedikit topik
β (beta / eta)	Distribusi kata per topik	β tinggi → topik berisi banyak kata berbeda; β rendah → topik fokus pada sedikit kata kunci
K	Jumlah topik	Hyperparameter yang harus ditentukan; pilih dengan coherence score

6.1.3 Inferensi

Karena posterior eksak intractable, LDA menggunakan metode aproksimasi:

- **Variational Inference:** optimasi parameter variational distribution untuk mendekati posterior.
- **Collapsed Gibbs Sampling:** MCMC sampling untuk mengestimasi distribusi topik.

6.1.4 Implementasi

```
from gensim import corpora, models

dictionary = corpora.Dictionary(processed_docs)
corpus = [dictionary.doc2bow(doc) for doc in processed_docs]

lda = models.LdaModel(corpus, num_topics=10, id2word=dictionary,
                      alpha='auto', eta='auto', passes=20)
for idx, topic in lda.print_topics(-1):
```

NER + IOB Application

"Joko Widodo mengunjungi kantor Perserikatan Bangsa-Bangsa di Jakarta pada 17 Agustus 2023."

Joko - B-PER, Widodo - I-PER, mengunjungi - O, kantor - O, Perserikatan - B-ORG, Bangsa-Bangsa - I-ORG, di - O, Jakarta - B-LOC, pada - O, 17 - B-DATE, Agustus - I-DATE, 2023 - I-DATE.

6.2 LSA (Latent Semantic Analysis)

LSA adalah pendekatan aljabar linear yang menemukan struktur laten dalam matriks term-document menggunakan Singular Value Decomposition (SVD). LSA mengurangi dimensi dan menangkap relasi semantik antar kata yang sering muncul bersama.

6.2.1 Mekanisme SVD

Matriks term-document A (m×n, m=jumlah term, n=jumlah dokumen) didekomposisi menjadi:

$$A = U \times \Sigma \times V^T$$

- **U** (m×k): matriks term-topic, merepresentasikan term dalam ruang topik.
- **Σ** (k×k): matriks diagonal singular values, urutan kepentingan topik.
- **V^T** (k×n): matriks topic-document, merepresentasikan dokumen dalam ruang topik.

Dengan mengambil k singular values terbesar (truncated SVD), kita memperoleh representasi dimensi rendah yang menangkap struktur semantik utama.

6.2.2 Implementasi

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD

tfidf = TfidfVectorizer(max_features=5000, stop_words='english')
X = tfidf.fit_transform(corpus)

lsa = TruncatedSVD(n_components=10, random_state=42)
X_lsa = lsa.fit_transform(X)
```

6.2.3 Kelebihan dan Kekurangan

- **Kelebihan:** matematika sederhana, dapat menangani sinonim hingga taraf tertentu, deterministik.
- **Kekurangan:** nilai pada matriks bisa negatif sehingga sulit diinterpretasi sebagai topik, tidak probabilistik, kurang fleksibel dibanding LDA.

6.3 NMF (Non-negative Matrix Factorization)

NMF adalah faktorisasi matriks dengan kendala bahwa semua entri matriks faktor harus non-negatif. Kendala ini menghasilkan representasi yang lebih mudah diinterpretasi dibanding SVD.

6.3.1 Formulasi

Matriks A (m×n, biasanya TF-IDF) didekomposisi menjadi dua matriks non-negatif W dan H:

$$A \approx W \times H, \quad W \geq 0, H \geq 0$$

- **W** (m×k): matriks bobot term-topic, kolom adalah topik dengan komposisi term.
- **H** (k×n): matriks bobot topic-document, kolom adalah dokumen dengan komposisi topik.

Optimasi dilakukan dengan meminimumkan $||A - WH||^2$ atau divergensi KL secara iteratif menggunakan multiplicative update rules.

6.3.2 Implementasi

```
from sklearn.decomposition import NMF

nmf = NMF(n_components=10, random_state=42, init='nndsvda')
W = nmf.fit_transform(X)  # bobot dokumen-topik
H = nmf.components_       # bobot topik-term

for i, topic in enumerate(H):
    top_words = [terms[j] for j in topic.argsort()[-10:][::-1]]
    print(f'Topic {i}: {top_words}')
```

6.3.3 NMF vs LDA vs LSA

Aspek	LDA	LSA	NMF
Pendekatan	Probabilistik	Aljabar linear (SVD)	Faktorisasi matriks
Non-negatif	Ya	Tidak	Ya
Interpretabilitas	Tinggi	Rendah	Tinggi
Stokastik	Ya	Tidak	Sebagian (init)
Cocok untuk korpus pendek	Kurang	Cukup	Baik

6.4 BERTopic

BERTopic adalah pendekatan modern yang menggabungkan kontekstual embeddings dari model Transformer (BERT atau Sentence-BERT) dengan teknik clustering untuk menghasilkan topik yang lebih koheren dan kontekstual.

6.4.1 Pipeline BERTopic

21. **Embedding:** hasilkan dense vector untuk setiap dokumen menggunakan Sentence-BERT atau model embedding lainnya.
22. **Pengurangan Dimensi (UMAP):** turunkan dimensi embedding (misal dari 768 ke 5–15) sambil mempertahankan struktur lokal dan global.
23. **Clustering (HDBSCAN):** kelompokkan embedding menjadi cluster berbasis density. HDBSCAN dapat mengidentifikasi outlier (noise) dan tidak memerlukan jumlah cluster predefined.
24. **Tokenization:** hitung representasi BoW per cluster.
25. **c-TF-IDF (class-based TF-IDF):** modifikasi TF-IDF di mana setiap cluster diperlakukan sebagai satu dokumen besar untuk mengekstrak kata kunci paling diskriminatif per topik.

6.4.2 Implementasi

```
from bertopic import BERTopic
from sentence_transformers import SentenceTransformer

embedder = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')
topic_model = BERTopic(embedding_model=embedder, language='multilingual')

topics, probs = topic_model.fit_transform(documents)
topic_model.get_topic_info()
```

6.4.3 Kelebihan BERTopic

- Memanfaatkan kontekstualitas BERT, sehingga topik lebih koheren secara semantik.
- Tidak memerlukan jumlah topik predefined (HDBSCAN menentukan otomatis).
- Mendukung dynamic topic modeling, hierarchical topics, dan visualisasi interaktif bawaan.
- Bekerja baik untuk Bahasa Indonesia dengan multilingual embedding seperti IndoBERT atau LaBSE.

6.5 Evaluasi Model Topik

6.5.1 Perplexity

Perplexity mengukur seberapa baik model memprediksi sampel data baru. Pada topic modeling, dihitung dari log-likelihood dokumen uji.

$$Perplexity = exp(-\Sigma \log p(w) / N)$$

Interpretasi: nilai lebih kecil lebih baik. Namun perlu dicatat bahwa perplexity rendah tidak selalu berkorelasi dengan kualitas topik secara human judgment.

6.5.2 Topic Coherence

Topic coherence mengukur seberapa semantik koheren kata-kata teratas di suatu topik. Beberapa varian populer:

- **UMass:** berbasis log-conditional probability dari kata-kata top.
- **UCI:** berbasis pointwise mutual information (PMI) di sliding window.
- **CV:** kombinasi dari indirect cosine similarity dan NPMI; paling populer dan berkorelasi baik dengan human judgment.

Interpretasi: nilai mendekati 1 lebih baik untuk c_v dan c_npmi. Untuk umass, nilai mendekati 0 (sebenarnya negatif tapi mendekati nol) lebih baik.

```
from gensim.models import CoherenceModel

cm = CoherenceModel(model=lda, texts=docs, dictionary=dictionary, coherence='c_v')
print(cm.get_coherence())
```

6.5.3 Ringkasan Metrik

Metrik	Arah Lebih Baik	Catatan
Perplexity	Lebih kecil	Tidak selalu mencerminkan kualitas topik bagi manusia
Topic Coherence (c_v)	Mendekati 1	Berkorelasi baik dengan penilaian manusia, paling direkomendasikan
Topic Diversity	Mendekati 1	Mengukur keunikan kata antar topik (mengurangi redundansi)